

HW Assignment 1: Implementing Apache Spark

Name: Olorunfunmi Shobowale

Date: September 28, 2026

Part 1 – Implementing Apache Spark and a Worked Example [40 pts]

1.1 What Apache Spark is

Apache Spark is an open-source, distributed engine for processing large datasets. Its main idea is to keep intermediate data in memory instead of writing it to disk after every step, which is what made Hadoop MapReduce slow for iterative jobs such as machine learning and interactive queries (Zaharia et al., 2016). A Spark application is run by a *driver* program that splits the work into tasks and hands them to *executors* through a *cluster manager*. The same code can run on one laptop (local mode) or on a cluster of hundreds of machines (Apache Software Foundation, 2026a).

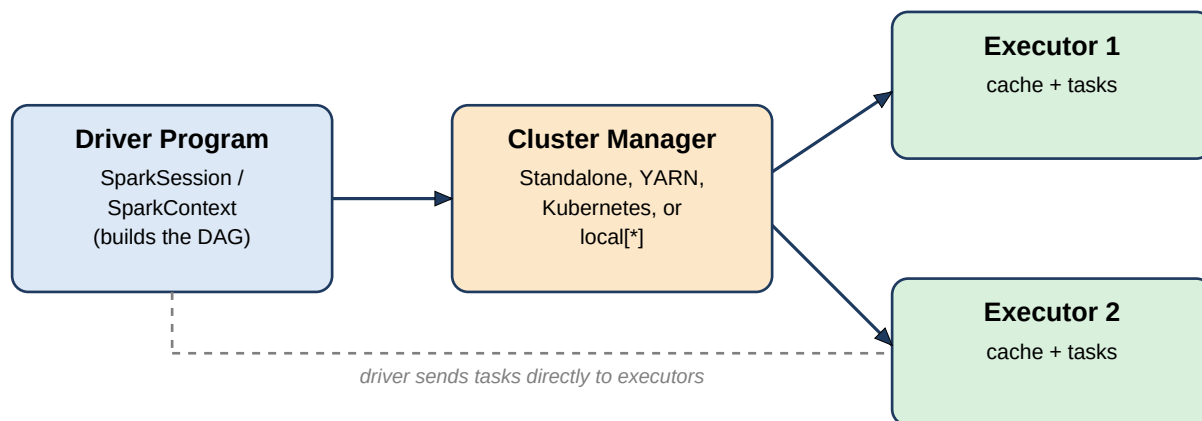


Figure 1. Simplified Spark runtime architecture (adapted from the Spark Cluster Mode Overview, Apache Software Foundation, 2026b).

1.2 My computing environment

Component	Version / setting
Operating system	Ubuntu 24.04.4 LTS (64-bit Linux)
Hardware	Intel Xeon @ 2.10 GHz, 4 CPU cores, 16 GB RAM
Java (JDK)	OpenJDK 21.0.10
Python	3.11.15 (in a virtual environment)
Apache Spark / PySpark	4.2.0, installed from PyPI
Deployment mode	Local mode, master = local[*] (uses all cores)

Table 1. Environment used for the implementation.

1.3 Implementation steps

I installed Spark in local mode, which is the standard way to develop and test Spark programs on a single machine. The commands I used were:

```
# 1. Confirm Java is present (Spark 4.x needs Java 17 or 21)
$ java -version
openjdk version "21.0.10" 2026-01-20

# 2. Create an isolated Python environment
$ python3 -m venv ~/spark-env
$ source ~/spark-env/bin/activate
(spark-env)$ pip install --upgrade pip setuptools wheel

# 3. Install Spark (the pyspark package ships the full Spark runtime + jars)
(spark-env)$ pip install pyspark
(spark-env)$ python -c "import pyspark; print(pyspark.__version__)"
4.2.0
```

Problem I hit during installation: installing `pyspark` directly into the system Python failed with *"Failed building wheel for pyspark"* because the operating system's bundled `setuptools/wheel` packages were too old and could not be upgraded (Ubuntu marks them as system-managed). **Solution:** I created a Python virtual environment, upgraded `pip`, `setuptools` and `wheel` inside it, and the installation then completed successfully. This is also a good habit in general because it keeps Spark's dependencies separate from the rest of the system.

1.4 Example problem and solution: Word Count

Problem. Given a text file, count how many times each word appears and list the most frequent words. Word count is the classic "hello world" of big-data processing because it uses the core map → shuffle → reduce pattern that larger jobs are built on. My input file `data.txt` contains five sentences about Spark.

Solution (PySpark code, `part1_wordcount.py`):

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("COSC611-WordCount").master("local[*]").getOrCreate()
spark.sparkContext.setLogLevel("ERROR")
print("Spark version:", spark.version)
print("Master:", spark.sparkContext.master)
print("Default parallelism:", spark.sparkContext.defaultParallelism)

lines = spark.sparkContext.textFile("data.txt")
counts = (lines.flatMap(lambda line: line.lower().split())
          .map(lambda word: (word, 1))
          .reduceByKey(lambda a, b: a + b)
          .sortBy(lambda kv: -kv[1]))
print("Total lines:", lines.count())
print("Top 8 words:")
for word, n in counts.take(8):
    print(f" {word:<12}{n}")
spark.stop()
```

How it works. `textFile` loads the file as an RDD of lines. `flatMap` breaks each line into words, `map` turns each word into a (word, 1) pair, `reduceByKey` adds up the 1s for each word (this is where Spark shuffles data between partitions), and `sortBy` orders the result. Nothing actually runs

until `count()` or `take()` is called, because Spark transformations are *lazy*.

Command and actual output:

```
(spark-env)$ python part1_wordcount.py
Spark version: 4.2.0
Master: local[*]
Default parallelism: 4
Total lines: 5
Top 8 words:
  spark      6
  data       4
  analytics  3
  in         3
  for        2
  big        2
  is         2
  apache     1
```

The output confirms Spark is correctly installed: the session started in local mode, used all four CPU cores (default parallelism = 4), read the file, and returned the correct counts ("spark" appears 6 times).

Part 2 – Tutorial: Preparing the Platform, Installing Spark, and Running Basic Operations [50 pts]

This tutorial walks a beginner from a clean computer to running Spark programs. Steps are shown for Linux/macOS, with notes for Windows where the steps differ.

Step 1 – Check system requirements

- **Memory:** at least 4 GB RAM (8 GB or more is more comfortable).
- **Java:** Spark runs on the Java Virtual Machine. Spark 4.x supports Java 17 and 21 (Apache Software Foundation, 2026a).
- **Python:** a recent Python 3 release (3.10 or newer) if you want to use PySpark.
- **Disk:** about 1 GB free for Spark, Java and sample data.

Step 2 – Install Java

```
# Ubuntu / Debian
$ sudo apt update && sudo apt install -y openjdk-21-jdk
# macOS (Homebrew)
$ brew install openjdk@21
# Windows: download and install "Eclipse Temurin JDK 21" from adoptium.net

# Verify on any OS
$ java -version
```

Set `JAVA_HOME` to the JDK folder if Spark later complains that it cannot find Java, e.g. `export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64` on Linux, or in *System Properties* → *Environment Variables* on Windows.

Step 3 – Install Python and create a virtual environment

```
$ python3 --version          # confirm Python 3.10+
$ python3 -m venv ~/spark-env # Windows: python -m venv %USERPROFILE%\spark-env
$ source ~/spark-env/bin/activate # Windows: %USERPROFILE%\spark-env\Scripts\activate
(spark-env)$ pip install --upgrade pip setuptools wheel
```

Step 4 – Install Apache Spark

Option A (simplest, used in this assignment): install PySpark from PyPI. The package includes the complete Spark engine, so no separate download is needed for local work.

```
(spark-env)$ pip install pyspark
```

Option B (full distribution): download a pre-built package from spark.apache.org/downloads, extract it, and add it to your `PATH`. This option also gives you `spark-shell` (Scala) and `spark-submit` for cluster jobs.

```
$ tar -xzf spark-4.x.x-bin-hadoop3.tgz
$ export SPARK_HOME=~/spark-4.x.x-bin-hadoop3
$ export PATH=$SPARK_HOME/bin:$PATH
```

Windows note: Spark on Windows also needs `winutils.exe` (Hadoop helper binaries). Put it in `C:\hadoop\bin` and set `HADOOP_HOME=C:\hadoop`. Without it, writing files fails with a "HADOOP_HOME is unset" error.

Step 5 – Verify the installation

```
(spark-env)$ python -c "import pyspark; print(pyspark.__version__)"
4.2.0
(spark-env)$ pyspark          # opens the interactive shell; type exit() to quit
```

If the version prints and the shell shows the Spark banner, the platform is ready. The warning *"Unable to load native-hadoop library"* is normal in local mode and can be ignored.

Step 6 – Prepare sample data

I created two small CSV files so every operation can be checked by hand:

```
sales.csv
order_id,region,product,quantity,unit_price
1,East,Laptop,2,950.00
2,West,Phone,5,610.00
3,East,Tablet,3,320.00
4,North,Laptop,1,980.00
5,South,Phone,4,590.00
6,West,Laptop,2,940.00
7,North,Tablet,6,300.00
8,East,Phone,3,620.00
9,South,Tablet,2,310.00
10,West,Tablet,4,315.00

regions.csv
region,manager
East,A. Johnson
West,B. Okafor
North,C. Williams
South,D. Adeyemi
```

Step 7 – Run basic Spark operations

The script `part2_operations.py` below performs seven basic operations, covering both of Spark's main APIs: the low-level RDD API and the higher-level DataFrame/SQL API. Every result shown was produced by actually running the script with `python part2_operations.py`.

```

from pyspark.sql import SparkSession, functions as F
spark = SparkSession.builder.appName("COSC611-BasicOps").master("local[*]").getOrCreate()
spark.sparkContext.setLogLevel("ERROR")

print("=== Op 1: Create an RDD and apply map / filter / reduce ===")
rdd = spark.sparkContext.parallelize(range(1, 11))
squares = rdd.map(lambda x: x * x)
evens = squares.filter(lambda x: x % 2 == 0)
print("squares:", squares.collect())
print("even squares:", evens.collect())
print("sum of squares:", squares.reduce(lambda a, b: a + b))

print("\n=== Op 2: Load a CSV file into a DataFrame ===")
df = spark.read.csv("sales.csv", header=True, inferSchema=True)
df.printSchema()
df.show(5)

print("=== Op 3: Select, add a column, and filter ===")
df2 = df.withColumn("revenue", F.col("quantity") * F.col("unit_price"))
df2.select("order_id", "product", "revenue").filter(F.col("revenue") > 1500).show()

print("=== Op 4: Group by and aggregate ===")
(df2.groupBy("region")
  .agg(F.sum("revenue").alias("total_revenue"), F.count("*").alias("orders"))
  .orderBy(F.desc("total_revenue"))
  .show())

print("=== Op 5: Join two DataFrames ===")
regions = spark.read.csv("regions.csv", header=True, inferSchema=True)
(df2.join(regions, on="region", how="inner")
  .select("order_id", "region", "manager", "revenue")
  .orderBy("order_id")
  .show())

print("=== Op 6: Spark SQL query ===")
df2.createOrReplaceTempView("sales")
spark.sql("""
  SELECT product, SUM(quantity) AS units, ROUND(AVG(unit_price),2) AS avg_price
  FROM sales GROUP BY product ORDER BY units DESC
""").show()

print("=== Op 7: Write results to disk (Parquet) and read back ===")
df2.write.mode("overwrite").parquet("output/sales_parquet")
back = spark.read.parquet("output/sales_parquet")
print("rows read back from Parquet:", back.count())
spark.stop()

```

Operation 1 – RDD transformations and actions

`parallelize` distributes a Python list across partitions. `map` and `filter` are *transformations* (lazy); `collect` and `reduce` are *actions* that trigger computation. Check: $1^2 + 2^2 + \dots + 10^2 = 385$, which matches.

```

squares: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
even squares: [4, 16, 36, 64, 100]
sum of squares: 385

```

Operation 2 – Reading a CSV into a DataFrame

With `inferSchema=True`, Spark scans the file and assigns proper types (integer, string, double). `printSchema` and `show` let you inspect the data.

```
root
 |-- order_id: integer (nullable = true)
 |-- region: string (nullable = true)
 |-- product: string (nullable = true)
 |-- quantity: integer (nullable = true)
 |-- unit_price: double (nullable = true)

+-----+-----+-----+-----+-----+
|order_id|region|product|quantity|unit_price|
+-----+-----+-----+-----+
|      1|  East| Laptop|       2|    950.0|
|      2|  West|  Phone|       5|    610.0|
|      3|  East| Tablet|       3|    320.0|
|      4|North| Laptop|       1|    980.0|
|      5|South|  Phone|       4|    590.0|
+-----+-----+-----+-----+
only showing top 5 rows
```

Operation 3 – Creating a column and filtering

`withColumn` derives a new revenue column ($\text{quantity} \times \text{unit_price}$), `select` keeps only chosen columns, and `filter` keeps orders above \$1,500. Six of the ten orders qualify.

```
+-----+-----+-----+
|order_id|product|revenue|
+-----+-----+-----+
|      1| Laptop| 1900.0|
|      2|  Phone| 3050.0|
|      5|  Phone| 2360.0|
|      6| Laptop| 1880.0|
|      7| Tablet| 1800.0|
|      8|  Phone| 1860.0|
+-----+-----+-----+
```

Operation 4 – Grouping and aggregation

`groupBy` + `agg` computes total revenue and order count per region. West leads with \$6,190 from 3 orders ($3050 + 1880 + 1260 = 6190$).

```
+-----+-----+-----+
|region|total_revenue|orders|
+-----+-----+-----+
|  West|      6190.0|      3|
|  East|      4720.0|      3|
|South|      2980.0|      2|
|North|      2780.0|      2|
+-----+-----+-----+
```

Operation 5 – Joining two DataFrames

An inner join on the `region` key attaches each region's manager to every order, just like a SQL JOIN.

```
+-----+-----+-----+-----+
|order_id|region|  manager|revenue|
+-----+-----+-----+-----+
|      1|  East| A. Johnson| 1900.0|
|      2|  West| B. Okafor| 3050.0|
|      3|  East| A. Johnson|   960.0|
|      4| North|C. Williams|   980.0|
|      5| South| D. Adeyemi| 2360.0|
|      6|  West| B. Okafor| 1880.0|
|      7| North|C. Williams| 1800.0|
|      8|  East| A. Johnson| 1860.0|
|      9| South| D. Adeyemi|   620.0|
|     10|  West| B. Okafor| 1260.0|
+-----+-----+-----+-----+
```

Operation 6 – Querying with Spark SQL

`createOrReplaceTempView` registers the DataFrame as a table, so ordinary SQL can be used. Spark's Catalyst optimizer turns the SQL and the DataFrame code into the same execution plan.

```
+-----+-----+-----+
|product|units|avg_price|
+-----+-----+-----+
| Tablet|   15|   311.25|
|  Phone|   12|   606.67|
| Laptop|    5|   956.67|
+-----+-----+-----+
```

Operation 7 – Writing and reading Parquet

Results are saved in Parquet, a compressed columnar format used widely in big-data systems, then read back to confirm all 10 rows were stored. Spark writes a folder containing part-files and a `_SUCCESS` marker.

```
rows read back from Parquet: 10
```

Step 8 – Monitor jobs and shut down

While a Spark application is running, the **Spark Web UI** at `http://localhost:4040` shows every job, stage, task, and cached dataset, which is the main tool for understanding performance. Always call `spark.stop()` at the end of a program to release memory and CPU.

Summary of operations demonstrated

#	Operation	API	Key methods
1	RDD transformations/actions	RDD	parallelize, map, filter, reduce
2	Load CSV data	DataFrame	read.csv, printSchema, show
3	Derive column & filter	DataFrame	withColumn, select, filter
4	Group & aggregate	DataFrame	groupBy, agg, sum, count
5	Join datasets	DataFrame	join
6	SQL query	Spark SQL	createOrReplaceTempView, sql
7	Save & reload	I/O	write.parquet, read.parquet

Part 3 – Challenges and Limitations of Implementing Apache Spark [10 pts]

Although Spark is powerful, my own setup and further reading showed several practical challenges:

- **Heavy dependence on memory.** Spark is fast because it keeps data in RAM, but that makes RAM the main cost and the main point of failure. If partitions do not fit in executor memory, Spark spills to disk (losing its speed advantage) or fails with out-of-memory errors. Sizing executor memory, cores and partitions correctly takes experience (Karau & Warren, 2017).
- **Installation and version compatibility.** Spark depends on matching versions of Java, Python/Scala, and Hadoop libraries. I experienced this directly: the install failed until I isolated it in a virtual environment. On Windows, the extra `winutils.exe` and `HADOOP_HOME` step is a common stumbling block, and Spark 4 no longer runs on Java 8 or 11.
- **Difficult tuning and debugging.** Because execution is lazy and distributed, an error in one line may only appear when an action runs much later, and stack traces mix Python and Java (JVM) messages. Performance problems such as data skew (one partition much larger than the others) or excessive shuffles require reading the Spark UI to diagnose.
- **Expensive shuffles.** Wide operations like `groupBy`, `join` and `reduceByKey` move data across the network. On a real cluster, shuffles are often the biggest bottleneck.
- **Not ideal for true real-time or small workloads.** Structured Streaming processes data in micro-batches by default, so latency is usually in the hundreds of milliseconds, higher than engines built for record-at-a-time streaming such as Apache Flink. For small datasets, Spark's start-up overhead makes it slower than plain pandas or a single database.
- **No storage layer of its own.** Spark is a compute engine only. It must be paired with HDFS, Amazon S3, a database, or a table format such as Delta Lake, which adds more components to install and secure.
- **Python overhead.** PySpark code that uses plain Python UDFs has to move data between the JVM and Python processes, which is slower than built-in DataFrame functions. Developers must learn to prefer built-in functions.
- **Cost and skills.** Production clusters (on-premises or cloud services like Databricks, Amazon EMR, or Google Dataproc) can be expensive, and teams need skills in distributed systems, cluster management and security (authentication, encryption, access control are not all enabled by default).

Conclusion. Spark was straightforward to set up in local mode once the environment was isolated, and it handled both RDD and DataFrame/SQL workloads correctly. Its real challenges appear at scale, in memory management, shuffles, tuning and operating a cluster. Knowing these limits helps decide when Spark is the right tool and when a simpler one is enough.

References

- Apache Software Foundation. (2026a). *Apache Spark documentation: Overview and downloading*.
<https://spark.apache.org/docs/latest/>
- Apache Software Foundation. (2026b). *Cluster mode overview*.
<https://spark.apache.org/docs/latest/cluster-overview.html>
- Apache Software Foundation. (2026c). *PySpark getting started: Installation*.
https://spark.apache.org/docs/latest/api/python/getting_started/install.html
- Apache Software Foundation. (2026d). *Spark SQL, DataFrames and Datasets guide*.
<https://spark.apache.org/docs/latest/sql-programming-guide.html>
- Karau, H., & Warren, R. (2017). *High performance Spark: Best practices for scaling and optimizing Apache Spark*. O'Reilly Media.
- Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M. J., Ghodsi, A., Gonzalez, J., Shenker, S., & Stoica, I. (2016). Apache Spark: A unified engine for big data processing. *Communications of the ACM*, 59(11), 56–65. <https://doi.org/10.1145/2934664>